Technical Project Specification

System Architecture & Technology Stack

The application will be engineered as a robust server-rendered web application utilizing Java, Spring Boot, and Thymeleaf. The chosen architecture balances backend maintainability with fast rendering and strong enterprise integration capabilities.

Core Stack Components

- **Primary Language:** Java (LTS Version)
- **Application Framework:** Spring Boot
    - **Spring MVC:** Request handling, routing, and controller layer management.
    - **Spring Security:** Authentication and authorization management.
- **Template Engine:** Thymeleaf
    - Natural HTML/HTML5 template integration for seamless UI rendering.
    - Dynamic layout dialect integration for reusable component modules.
- **Build System:** Gradle / Maven standard build lifecycle.

Testing Framework & Integration Testing Requirements

A mandatory requirement for this project is rigorous Integration Testing (IT). Integration tests must validate application context loads, database transactions (if applicable), external system web clients, and HTTP routing endpoints.

Integration Testing (IT) Guidelines

- **Test Framework:** JUnit 5 with `@SpringBootTest` annotation context.
- **Mocking & Assertion:** Mockito for isolation when needed; AssertJ for fluent assertions.
- **MockMvc:** Full testing of controller endpoints, form submissions, and rendered Thymeleaf view names and model attributes without running a full HTTP server.
- **Test Automation:** The CI/CD pipeline must execute the `integrationTest` phase prior to any deployment trigger. Builds failing any IT suite must halt immediately.

Infrastructure Integration & Environmental Discovery

The deployment infrastructure is pre-provisioned. The development process relies on inspective environment analysis rather than assuming static configuration standards.

Environment Component	Discovery Task	Integration Strategy
Reverse Proxy / Load Balancer	Check header forwarding (`X-Forwarded-For`, `X-Forwarded-Proto`).	Configure `server.forward-headers-strategy=native` or `framework` to handle static assets and URL redirects correctly.
Base Paths & Routing Context	Inspect target URI prefixes and ingress paths.	Adjust `server.servlet.context-path` dynamically or configure relative URL handling within Thymeleaf standard expressions (`@{/}`).
Asset CDN & Static Resources	Identify static asset paths and caching proxy rules.	Ensure Thymeleaf correctly maps static CSS, JS, and image assets relative to deployment context.
Environment Variables	Query active profile flags and runtime secrets.	Bind platform configs to `@ConfigurationProperties` classes using standard Spring conventions.

Phased Development & Verification Strategy

To ensure zero-downtime deployment stability and confirm infrastructure compatibility, execution must proceed through a strict sequential deployment flow.Phase 1: Canary/Test Page Deployment

1. Create a lightweight standard controller (`TestController.java`).
2. Implement a minimal Thymeleaf template (`test-index.html`) displaying essential environment diagnostics (e.g., active profile, context path, uptime, base styling).
3. Push to repository and trigger deployment pipeline.

Phase 2: Automated Verification & Infrastructure Inspection

1. **Pipeline Await:** The execution process must pause until deployment health checks yield successful returns.
2. **Behavior Verification:** Verify HTTP 200 status code response and check response headers.
3. **Infrastructure Analysis:** Inspect how assets are served and confirm that path transformations perform correctly in the deployed state.

Phase 3: Core Application Implementation

1. Upon successful verification of the test page, proceed with full application development.
2. Develop domain entities, repository layers, business logic services, and rich UI view templates.
3. Enforce mandatory Integration Test coverage prior to full application release.